

DocuRAG Refactor Notes

Updates to the Intelligent PDF Question-Answering System and how the new architecture works.

1. Summary of Updates

The project was reorganised around a single reusable engine and three thin frontends. Along the way, several latent bugs surfaced and were fixed, the LLM backend was standardised on Gemini, and the embedding model was moved off the local machine and onto the Hugging Face Inference API.

Functional changes

- Removed Mistral entirely; Gemini is the only LLM in the codebase.
- Embeddings now run through the Hugging Face Inference API (**HuggingFaceEndpointEmbeddings**), so the **BAAI/bge-m3** model is never downloaded locally.
- Chroma is imported from **langchain-chroma** (the **langchain-community** version is removed in LangChain 1.0).
- Answers now carry the source documents they were built from, so the Streamlit app can display page citations underneath each answer.

Structural changes

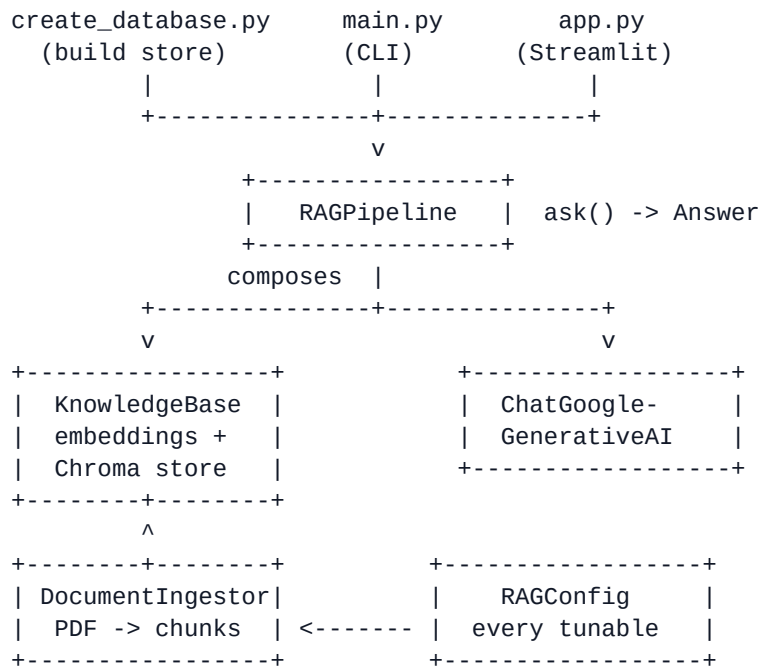
- All logic moved into a reusable **rag/** package with four classes: **RAGConfig**, **DocumentIngestor**, **KnowledgeBase**, **RAGPipeline**.
- **create_database.py**, **main.py**, and **app.py** collapsed into thin frontends over that engine, so the CLI and the web app can no longer drift apart.
- A **tests/** suite was added: 31 pytest tests with 100% branch coverage of the **rag/** package, runnable without any API keys thanks to dependency injection.
- Continuous integration via GitHub Actions across Python 3.11-3.14.

Bugs fixed along the way

- **ChromaDb** vs **ChromaDB** case mismatch between **main.py** and **create_database.py** (Linux case-sensitive) — the retriever was opening an empty database.
- **response.text** vs **response.content** on Gemini responses — **.text** is a method on **AIMessage**, not the string.
- **os.path.exists(chroma_db)** returned **True** for an empty directory, so the Streamlit Q&A box would appear over an empty store — replaced with **KnowledgeBase.is_populated**, which checks for actual rows.
- **gemini-3.6-flash** is not a real Google model id (there is no public 3.6-flash release); the model id is now centralised in **RAGConfig**, so switching to **gemini-2.5-flash** or **gemini-2.0-flash** is a one-line change.

2. Architecture

The **rag/** package holds every piece of logic. The three scripts at the project root are thin frontends that share the same engine, wired together by dependency injection.



Class responsibilities

Class	Responsibility
RAGConfig	Frozen dataclass holding every tunable: LLM model, embedding model, persist directory, chunk size and overlap, retrieval settings.
DocumentIngestor	Loads a PDF with PyPDFLoader and splits it into overlapping chunks. Config-driven, so the same instance can be reused across many PDFs.
KnowledgeBase	Owns the embedding client and the Chroma store. Lazy-opens the store on first use, exposes an MMR retriever, and reports whether the store actually has rows via is_populated .
RAGPipeline	Composes a retriever, a prompt and an LLM into ask() . Constructor takes retriever and LLM as arguments (dependency injection); from_config() wires the real ones.
Answer	Immutable result carrying the text, the source documents, and a .pages property that returns the sorted, de-duplicated list of PDF pages the answer drew from.

3. Design Ideas Made Concrete

Encapsulation

Callers never touch **Chroma** or the embedding client directly. **KnowledgeBase** owns both, opens the store lazily on first access, and exposes only what the rest of the code needs: **build()**, **as_retriever()**, **is_populated**.

Composition over inheritance

RAGPipeline *has* a retriever and *has an* LLM rather than inheriting from either. Swapping the LLM for another provider is a constructor argument, not a subclass.

Dependency injection

The pipeline takes its collaborators as arguments:

```
RAGPipeline(retriever=..., llm=...)
```

The **from_config()** factory classmethod wires the real ones for production. The test suite passes fakes instead:

```
pipeline = RAGPipeline(
    retriever=FakeRetriever(documents),
    llm=FakeLLM(reply="42"),
)
```

This is what lets the whole engine run in tests without an API key, without a network, and without Chroma persisting anything on disk.

Single source of truth

Every tunable lives in **RAGConfig**. Frontends override only what they need; the Streamlit app overrides one field so its uploads never mix with the corpus built by **create_database.py**:

```
CONFIG = RAGConfig(persist_directory="chroma_db")
```

4. How Each Frontend Uses the Engine

create_database.py — build the vector store

```
config = RAGConfig()
chunks = DocumentIngestor(config).ingest(PDF_PATH)
KnowledgeBase(config).build(chunks)
```

Three lines. Loads the PDF, splits it into overlapping chunks, embeds those chunks through the Hugging Face Inference API, and persists the vectors into Chroma.

main.py — command-line chat

```
pipeline = RAGPipeline.from_config()
while True:
    query = input("you :")
    if query == "0":
        break
    answer = pipeline.ask(query)
    print(f"\n Ai:{answer.text}")
```

The old CLI constructed its own embedder, its own Chroma handle, its own prompt, its own retriever, and its own LLM. All five now live inside **RAGPipeline.from_config()**.

app.py — Streamlit web app

The web app caches the pipeline with **@st.cache_resource** so Gemini isn't reconstructed on every keystroke, and clears that cache after a new upload. It also now shows source pages under each answer — a feature the old app couldn't offer because it didn't track sources.

```
answer = get_pipeline().ask(query)
st.write(answer.text)
if answer.pages:
    st.caption(f"Source pages: {' , '.join(str(p) for p in answer.pages)}")
```

5. The Test Suite

Because **RAGPipeline** and **KnowledgeBase** take their collaborators as constructor arguments, the entire suite runs against fakes: **31 tests, 100% branch coverage of the rag/ package, under one second, with no API keys and no network calls.**

```
pip install -r requirements-dev.txt
pytest                # run the suite
pytest --cov          # with a coverage report
```

Layout

```
tests/
|- conftest.py          shared fixtures and offline test doubles
|- test_config.py       RAGConfig: defaults, immutability, overrides
|- test_ingestion.py    DocumentIngestor: chunking, overlap, real PDF
|- test_knowledge_base.py KnowledgeBase: build, persist, reopen, retrieve
+- test_pipeline.py     RAGPipeline + Answer: prompt, sources, pages
```

Test doubles

conftest.py supplies three deterministic, offline doubles:

Double	Replaces	Why
HashEmbeddings	Hugging Face Inference API	Real embeddings need a token and are not reproducible across runs.
FakeLLM	Gemini	Records the prompt it receives, so tests can assert that retrieved context actually reached the model.
FakeRetriever	Chroma retriever	Returns a fixed document set and records the question it was asked.

What is covered

- Retrieved chunks actually reach the prompt, separated correctly.
- The prompt carries the instruction to refuse questions outside the context.
- **Answer.pages** is sorted and de-duplicated; **Answer** and **RAGConfig** are immutable.
- Config values propagate into the retriever's search settings.
- Chunking honours size, overlaps between neighbours, and preserves page metadata.
- A real two-page PDF is generated and loaded, exercising **PyPDFLoader** rather than assuming it.
- Full Chroma round trip: build → persist → reopen → retrieve.
- An empty directory does not count as a populated store.
- **retrieve()** does not call the LLM; **ask()** calls it once.

Markers

Tests that write a real Chroma store to a temporary directory are marked **integration** (still offline, still using **HashEmbeddings**), so the fast unit tests can be run alone:

```
pytest -m "not integration"    # 24 unit tests, 0.5s
pytest -m integration          # 7 storage round-trip tests
```

Strictness and CI

pyproject.toml sets **filterwarnings = error**, so a new deprecation from LangChain or Chroma fails the build instead of scrolling past; known third-party warnings are allow-listed individually with a comment explaining each. GitHub Actions runs the suite on every push and pull request across Python 3.11 to 3.14, with **no secrets configured on purpose** — if a test ever starts needing an API key, the build breaks, which keeps the suite honest.

6. Running the Project

Environment variables

Two keys must be present in `.env` at the project root:

```
GOOGLE_API_KEY=your_google_api_key
HUGGINGFACEHUB_API_TOKEN=your_hugging_face_token
```

- **GOOGLE_API_KEY** — used by Gemini to generate answers.
- **HUGGINGFACEHUB_API_TOKEN** — used to call the Hugging Face Inference API for embeddings. Get one at huggingface.co/settings/tokens.

Commands

```
# Install dependencies
pip install -r requirements.txt

# Build the vector store from the bundled PDF
python create_database.py

# Command-line chat
python main.py

# Web app
streamlit run app.py

# Tests (no API keys required)
pytest
```

If Gemini rejects the model id

gemini-3.6-flash is not a public Google model id at the time of writing. If the first live query returns a 404, open **rag/config.py** and change one line:

```
llm_model: str = "gemini-2.5-flash"    # or "gemini-2.0-flash"
```

Because the model id is centralised in **RAGConfig**, this is a single edit rather than three.